Classical Reasoning for Knowledge Base Entailment

AI Assignment Report

Arushi Gandhi | Roll No. M24MA2001

1. Task and representation

Our task is to decide whether a query follows from a set of facts and rules. The final True or False label comes from the reasoning engines. Since the evaluation input will already be in logic, the parser and inference code are considered the main parts of the system. RuleTaker and CounterBench still need an English adapter for the public experiments. That adapter uses fixed patterns.

The terms have been stored as tagged tuples. For example, (v, x) is a variable, (c, alice) is a constant, and (f, name, arguments) is a function term. Formulae are trees whose nodes hold predicates, connectives or quantifiers. Before inference, the reasoning engines work with literals of this form (sign, predicate, arguments). This makes substitution as well as unification possible without changing the original input text.

The parser accepts Unicode notation, ASCII operators and Prolog clauses. Operator precedence runs from negation through conjunction, disjunction, XOR, implication and biconditional. Implication groups to the right. Free variables x, y, z, u, v, w and single capital letters are universally closed; Prolog clauses also allow longer capitalised variable names. The parser checks the complete input, including brackets. A malformed formal premise gives PARSE_ERROR.

Names need careful handling. Ready1 and Ready2 must stay different, and Student must not silently become Students. The formal parser preserves these distinctions. For the FOLIO experiment, an optional annotations profile repairs bracket imbalance and normalises some spelling and predicate variants. These types of repairs can change the meaning of the formula. They are, however, kept out of the default formal-input path.

The shape of rule does not determine its semantics. The premises are P(alice) and not Q(x) implies R(x). Under classical entailment, the absence of Q(alice) is insufficient to infer R(alice). A closed-world program can infer it. Classical semantics are used by default and that require an explicit setting for closed-world rules [1].

2. Reasoning algorithms

2.1 Rule evaluation and reusable knowledge bases

Several queries share the same premises. compile_kb stores the parsed formulas, clauses, rule indices and reusable answers, with a cache limit of 2,048 knowledge bases. Backward chaining can reuse completed answers. If a loop or depth cutoff interrupts the search, the partial memo is discarded and then query moves to tabled evaluation.

Agenda of the new facts are kept by tabled evaluation. Each fact triggers rules indexed under its predicate, and repeated answers are ignored. Recursive relations can then reach a fix point without a proof-depth cutoff. For closed-world negation, alternating upper and lower estimates identify the true, false and undefined atoms [3]. An undefined atom keeps the status UNKNOWN.

2.2 CNF, SAT and equality

Direct CNF conversion is appropriate for small formulas. For larger expressions, an auxiliary predicate is introduced for each sub-formula and clauses are added defining it. Its arguments are the free variables of that sub-formula. This avoids distributing every conjunction and disjunction. XOR and biconditional each need four defining clauses. Variables are standardised apart as well as existential quantifiers introduce fresh Skolem terms.

The SAT engine uses DPLL and watched literals [4]. It checks the premises together with the negated query. Function free Skolem clauses are grounded over their constants and at the same time a fresh constant supplies a nonempty domain when needed. Ground formulas with uninterpreted functions are also handled. When variables and functions occur together, an unsatisfiable set of ground instances proves the query, whereas a satisfiable sample may leave other instances unchecked.

For ground equality, congruence closure groups terms that must have the same value. SAT assignment that breaks equality or predicate congruence is excluded by a theory lemma, but the search is continued. General resolution receives equality axioms also including substitution. Before a long first-order search, a bounded model check tries domains of one or two elements. A valid countermodel establishes non-entailment. If neither size works, proof search is continued.

2.3 Resolution and causal inference

Firstly, resolution tries a relevant subset of the premises, then falls back to the full KB. The full search matters when an unrelated contradiction makes every query entailed. The prover uses an occurs check, factoring, subsumption, unit preference and renamed self-resolution [2]. Clause comparisons are reduced by indexes and duplicate filtering. The full fallback allows 1,200 selected clauses and 0.3 seconds; input depth and length are retained. Counterfactual queries use structural equations [8]:root assignments are retained by the causal engine under its observation policy, applies the intervention and propagates values. A unique Boolean fixed point are required by small unresolved cycles.

3. Experiments and final evaluation

All three public datasets are used. RuleTaker uses closed-world semantics [5]. FOLIO uses its FOL annotations, classical entailment and the annotations profile [6]. Its True label maps to True; both False and Uncertain map to False for this binary entailment task. CounterBench uses the retained hybrid observation policy [7]. There is no separate held-out CounterBench split in this evaluation, so that score needs caution.

Dataset	Queries	Correct	Accuracy	Inference s
RuleTaker test	151911	151911	100.0000%	2.904884
FOLIO validation	203	176	86.6995%	5.439005
CounterBench	1200	1135	94.5833%	0.022385
All	153314	153222	99.9400%	8.366274

Table 1. Final results from a fresh execution of the submitted notebook.

The mean of the three dataset accuracies is 93.7609%. The combined accuracy in Table 1 is much higher because RuleTaker supplies most of the queries. Inference timing includes compilation when a knowledge base is first encountered. Repeated contexts can reuse cached work, and caches are cleared between datasets. English conversion took another 15.231130 seconds. The table rounds times to six decimal places; the notebook keeps the full values.

Dataset	Accuracy	Before median s	After median s
RuleTaker test	100.0000%	3.314211	2.943720
FOLIO validation	86.6995%	7.429806	5.398415
CounterBench	94.5833%	0.021917	0.022350

Table 2. Earlier speed revision: per-dataset medians of three runs per version.

In the earlier three-run speed comparison, median total inference fell from 10.765704 to 8.329302 seconds, a reduction of 22.63%. All 153,314 public predictions stayed the same in each round. The changes reused completed backward answers, cached term depths and reduced repeated resolution work. Search budgets and semantics stayed the same. These laptop timings can vary between runs, and some individual queries may take longer. FOLIO train accuracy was 807/1001.

4. Examples of reasoning traces

The following examples show how the engines reach an answer. The notebook also prints the corresponding rule applications, refutations and causal assignments.

4.1 Rule derivation

For the assignment example, the facts are Young(bob) and Kind(bob). The rules say that young individuals are smart, and that a smart, kind individual is successful. To answer Successful(bob), the tabled engine builds this derivation:

```
Successful(bob) by rule [Smart(x) and Kind(x) -> Successful(x)]
  Smart(bob) by rule [Young(x) -> Smart(x)]
    Young(bob) (fact)
  Kind(bob) (fact)
Result: True, PROVED
```

4.2 Resolution refutation

Take P(alice) or Q(alice), together with not P(alice), and ask whether Q(alice) follows. Resolution adds negation Q(alice). The empty clause at the end shows that the premises and negated query cannot all hold:

```
1. P(alice) or Q(alice)          [premise]
2. not P(alice)                 [premise]
3. not Q(alice)                 [negated query]
4. Q(alice)                     [resolve 1, 2]
5. empty clause                [resolve 4, 3]
Result: True, PROVED
```

4.3 Equality and a countermodel

Given alice = bob and P(f(alice)), the engine can infer P(f(bob)). Firstly, alice and bob are merged; congruence then merges f(alice) and f(bob). Assigning different truth values to the two P atoms would violate that equality. A different situation is Ready1(alice) with query Ready2(alice): the first predicate can be true while the second is false. Preserving the two names is necessary for this countermodel.

4.4 Counterfactual

```
B := A       C := B
Query: do(not A) | A => not C
Factual root: A = 1, hence B = 1 and C = 1
Intervene: A = 0, hence B = 0 and C = 0
Target not C holds in every retained world.
Result: True, PROVED
```

A negative rule cycle needs a separate interpretation. With P if not Q and Q if not P, well-founded evaluation leaves both atoms undefined. Their Boolean output is False, while their internal status is UNKNOWN. The status remains available so that the output is not mistaken for a completed classical countermodel.

5. Learned advisor experiments

Which classical search stage to run first and which clauses to select early are also tried to learn. These advisors never decide answer label. Their training data came from FOLIO train and then it generated development templates, grouped by story or template for the internal split. Gold entailment labels were not training targets. From 1,481 attempted development problems, the collector obtained 508 checked refutations and 16,471 clause examples.

The clause scorer is a regularised logistic model with twelve structural features, fitted with SGD. Clauses used in a checked refutation provide positive targets; other sampled clauses from the same search provide negative targets. Predicate names and example ids are excluded at the same time. The router is a cost-sensitive tree with thirteen features and solver runtime as its target. Features outside the fitted range, uncertain leaves and invalid actions return the usual schedule. Related clause-selection work includes ENIGMA [9].

Guidance	FOLIO train s	FOLIO validation s
off	13.598945	5.239496
router	13.039181	5.359992
ranker	14.216936	5.511066
learned	13.020787	5.359625

Table 3. Single-run inference times. Every mode gives 807/1001 correct on FOLIO train and 176/203 on validation.

The result was mixed. Routing reduced inference time on FOLIO train, but that advantage did not carry over to the validation run. Forcing an extra guided clause search was slower and corrected no additional answers. They remain available with guidance=router, ranker or learned; auto uses the classical schedule. Since public validation helped select this setting, it is not an untouched evaluation.

Guided resolution has a derivation checker. It checks the input clauses, resolution and factoring steps, and the empty root clause before accepting a refutation. An unsuccessful guided probe leaves the original fallback budgets in place. Training refutations are also checked, and verify=True enables checking for an ordinary resolution run. This work also corrected a variable-renaming collision: distinct variables must stay distinct after renaming.

6. Limitations and running the notebook

First-order search has time, clause, grounding and term-depth limits, so it can miss a valid proof. PROVED records an entailment proof. DISPROVED records a completed negative decision from a supported procedure. UNKNOWN, LIMIT_REACHED, PARSE_ERROR and UNSUPPORTED remain in the diagnostics. The required CSV maps all unresolved results to False, so an unresolved entailed query still counts as an error.

Input interpretation can also lose accuracy. The FOLIO annotation repairs are useful for that dataset, but they can merge names that should be distinct in clean logic. An earlier fuzzy matching approach even confused satisfactory with unsatisfactory. The names, however, are kept unchanged by the default parser. The chosen world assumption matters too: a Horn clause can occur in either a classical theory or a closed-world program.

The earlier Z3 comparison shared the parser, so it could not isolate annotation errors from parser errors. The current FOLIO score is not an accuracy ceiling. Quantified equality still relies on general axioms rather than a dedicated superposition procedure. Causal enumeration can also grow quickly; the cycle fallback allows at most sixteen endogenous variables. Its observation policy is the one retained for CounterBench.

The submission folder contains four files: the notebook, this report, the CSV and the README. The evaluation workflow needs pandas, datasets, huggingface_hub and Jupyter. The reasoning engines use the Python standard library.

References

[1] Russell and Norvig. Artificial Intelligence: A Modern Approach, 4th edition, chapters 7-9.

[2] Robinson. A machine-oriented logic based on the resolution principle. JACM 12(1), 1965.

[3] Van Gelder. The alternating fixpoint of logic programs with negation. PODS, 1989.

[4] Davis, Logemann and Loveland. A machine program for theorem-proving. CACM 5(7), 1962.

[5] Clark, Tafjord and Richardson. Transformers as soft reasoners over language. IJCAI, 2020.

[6] Han et al. FOLIO: Natural language reasoning with first-order logic. EMNLP, 2024.

[7] CounterBench/CounterBench. Public Hugging Face dataset, balanced alpha V1 and backdoor V2.

[8] Pearl. Causality, 2nd edition, chapter 7, 2009.

[9] Jakubuv and Urban. ENIGMA: Efficient learning-based inference guiding machine. arXiv:1701.06532, 2017.